

A* GPS Road Navigation System

By William L.

Problem Statement and Approach sections written by Jude L. and Matthew B.

California State University, Fullerton

CPSC - 481-04 17357

Dr. Hamid Ebrahimi

December 12 2025

Problem Statement

The main problem we discussed in this project is the challenge of long travel time and inefficient navigation when determining routes across a city. Most of the road networks contain thousands of interconnected streets and intersections, which creates a large and complex search space for any navigation system. Without an efficient search algorithm, the system may need to explore a lot of possible routes before identifying the best one.

Traditional uniform search methods, such as UCS, will always find the optimal path, but they often do so by examining many unnecessary roads and intersections. This leads to longer computation time, especially for a large map, just like Fullerton. As a result, a navigation system relying solely on UCS can struggle to quickly route, making it impractical for real-time use. We aim to solve this problem by creating a navigation system that can compute fast, accurate, and real-time optimal routes. To achieve this, we explored an informed search algorithm A* that uses a heuristic to guide the search more intelligently. By reducing the number of nodes explored and focusing the search toward the destination, we sought to improve efficiency without sacrificing accuracy.

Approach

To solve this problem, we collected map data from OpenStreetMap of the city of Fullerton. Following that, we created a tool that would navigate this map using both A* and UCS. This allows us to have a side-by-side comparison of how A* and UCS perform, both in terms of efficiency and optimality.

Software Description

The navigator consists of a roadmap, a roadmap maker, and a utility class. The roadmap class instance is constructed from edges and nodes of different types (e.g. traffic control, junction, dead-end, etc), and also contains the implementation of the A* and UCS algorithms as class member functions. The cost function $g(x)$ (or in our program it is the function `RoadMap.road_cost`) per road/edge is an estimate of the time in hours it would take to travel on

that road given the length of the road in miles divided by a speed estimate in miles per hour. The roadmap maker reads the map data and constructs a roadmap. Our heuristic (RoadMap.heuristic) is calculated by finding the euclidian distance from the end of a road to the destination node in miles divided by a running average speed limit of all roads previously added to the frontier. The function draw_path in utility.py produces an image of the map with the optimal path drawn on.

At runtime, the main file creates the map, selects a random starting point and destination, and runs both algorithms, benchmarking their execution time and estimated travel time. This way, we can compare the results and performance of the algorithms. Following that, it produces an image of each algorithm's path that also displays the runtime and estimated travel time.

Findings

In our project, we evaluated the performance of the A* search algorithm compared to UCS using real map data from OpenStreetMap. Our tests demonstrate that A* always benchmarked faster and in rare cases was only slightly less optimal than UCS in **63.33%** of tests when using our scaled euclidean distance heuristic.

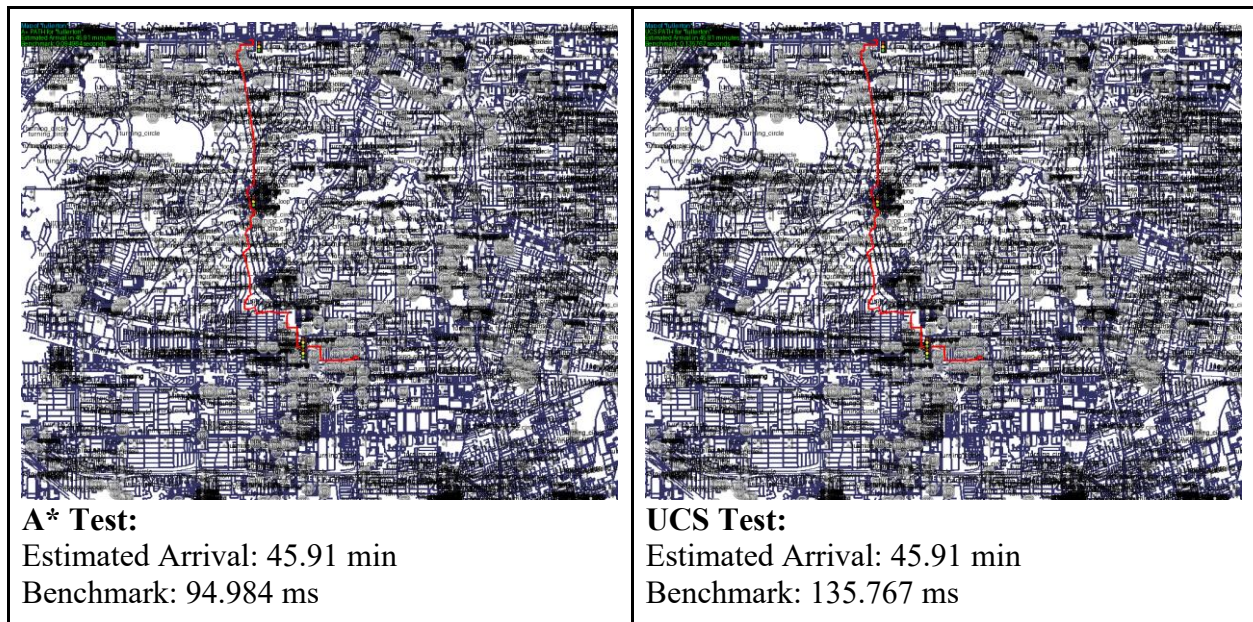
While both A* and UCS are complete and optimal, UCS is uninformed; it expands nodes solely based on cost so far, without any knowledge of where the goal is located. This leads UCS to unnecessarily explore large portions of the map. A* improves efficiency by combining $g(n)$ - the cost so far- and $h(n)$ - the estimated cost to the goal. This allows A* to focus its search direction, avoid unnecessary breaches, and reach the goal with far fewer node expansions. Across all the test cases from our presentation, A*, in **63.33%** of test cases, produced lower benchmark times, preserving almost all optimality. The following table shows the test data we obtained across 180 test cases and their results alongside an image comparison of A* vs UCS.

	Average Time-to-Arrival (path cost) (min)	Average Benchmark (ms)	Runtime Difference (ms)
A*	28.158857	88.408	0.727
UCS	28.138209	87.682	

In **63.33%** (a majority) of tests where A* benchmarked faster times than UCS:

	Results in Tests Where A* Benchmarked Faster
Average Benchmark Difference	A* faster by 20.514 ms
Average Time-to-Arrival Difference (path cost)	UCS faster by 0.014604 min

Here is a render of a single A* test (left) and the equivalent UCS test (right):



Conclusion and Future Work

What we've learned is that A* given our heuristic is demonstrably faster than UCS a majority of the time with virtually no loss in optimality. For future developments, we would like to be able to get real-time traffic data to ensure that the accuracy of the algorithm remains consistent as traffic conditions change. Additionally, we would like to come up with a more sophisticated heuristic than Euclidean distance to improve the optimality of the algorithm.

Reference

Dataset: OpenStreetMap- <https://download.geofabrik.de/north-america/us/california.html>

Libraries:

- PIL- For image processing

- Pyrosm- For reading OpenStreetMap data
- Pandas- For data processing
- GeoPandas- Allows Pandas to work with geospatial data
- Shapely- For geometry tools